arXiv:2611.11533v1 [cs.DC] 8 Nov 2026

OSTINATO: Fast CPU–GPU MoE Inference for Local Agents via Resident Experts in Every Forward Pass

Junning Zhu [1]

Abstract

Language-model agents increasingly run on workstations with a few consumer GPUs, where large mixture-of-experts (MoE) models keep part of their experts in host memory. With existing CPU–GPU engines, these agents get fast prefill or fast decode, not both: llama.cpp's expert cache skips forward passes of more than 32 tokens to keep the experts of decode cached, so an agent waits 2.1–2.8 s after each tool output, whereas its default layer placement shortens this wait but is 1.5–1.6× slower in decode. Our key insight is that a short prefill touches too many distinct experts to upload, yet sends 84% of its expert activations (*routes*) to a stable set of experts that fits in GPU memory. We present OSTINATO, a CPU–GPU MoE inference engine that computes these experts on the GPUs in every forward pass without changing what the model computes. Short prefills split their routes, leaving fewer than one of a token's six routed experts per layer to the CPU; long prefills upload each distinct cold expert once; and each decode step runs as one fixed-shape CUDA graph per GPU. On real coding and customer-service agent sessions with DeepSeek-V4-Flash on four RTX 4090D GPUs, OSTINATO is the fastest engine in both phases and finishes every session sooner than all six baselines. Against llama.cpp and KTransformers, it cuts the median wait after a tool output by 22–55% and finishes sessions 1.4–2.5× sooner.

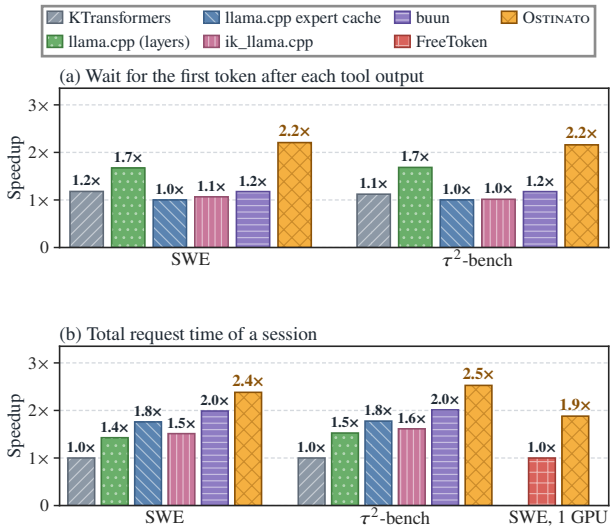

Figure 1. OSTINATO is the fastest engine both in the wait after each tool output and in the total request time. Its median wait is 1.26 s on SWE and 0.98 s on τ^2-bench, 46–55% below those of the expert caches and KTransformers and 22–24% below that of llama.cpp's layer placement, and it finishes sessions 1.2–2.5× sooner than every baseline. Bars: speedup over the slowest engine in each group (over FreeToken on one GPU) on DeepSeek-V4-Flash with four RTX 4090D GPUs, replaying held-out coding (SWE) and customer-service (τ^2-bench) agent sessions.

1. Introduction

Language-model agents, which repair software repositories (Yang et al., 2024; Wang et al., 2025) or serve customers through tools (Barres et al., 2026), increasingly run on workstations that their users own (Chen et al., 2025; Yang et al., 2026a; The ggml authors, 2026). Running an agent locally keeps its repository, its records and every tool output on the user's machine and lets one open-weight model serve all turns of a long session, thereby supporting agents for private code and data. Among open-weight models, mixture-of-experts (MoE) architectures (Shazeer et al., 2017; Fedus et al., 2022) now underlie many of the most capable ones (DeepSeek-AI, 2024; Yang et al., 2025; DeepSeek-AI, 2026a), and recent releases such as DeepSeek-V4-Flash emphasize agentic capabilities (DeepSeek-AI, 2026b). By activating only a few of its many experts for each token, an MoE model grows its capacity while keeping the computation per token small. However, the experts of a large MoE model exceed the memory of a few consumer GPUs. The routed experts of DeepSeek-V4-Flash take 137 GiB, of which only about half fit on four 24 GB GPUs beside the dense weights and the key–value (KV) cache, so local engines keep the rest in host memory (Kamahori et al., 2025; Chen et al., 2025; The ggml authors, 2026; Yang et al., 2026a).

To reduce expert traffic in memory-constrained MoE infer-

[1]Beijing Normal-Hong Kong Baptist University. Correspondence to: Junning Zhu <t330025113@mail.bnbu.edu.cn>.

Preprint. November 10, 2026.

ence, one line of work changes what the model computes. Such methods compress experts (Zhou et al., 2025b), redesign or fine-tune the router so that consecutive tokens reuse experts (Zhou et al., 2025a; Zhu et al., 2026b), or reroute tokens toward experts that other tokens in a batch already load (Oncescu et al., 2026). While effective in reducing expert loads, these changes alter the tokens that the model generates, so each method has to re-establish quality on downstream benchmarks; FloE, for example, reports an average degradation of 4.4–7.6% on downstream tasks (Zhou et al., 2025b). For an agent, whose next tool call depends on every token it generates, these findings favor an inference engine that leaves the model's computation unchanged, particularly when the agent acts without a person checking each step.

An alternative line of work leaves the model unchanged, holding some experts in GPU memory and the rest in host memory (Kamahori et al., 2025; Cao et al., 2025; Chen et al., 2025; Zhong et al., 2025; The ggml authors, 2026). When a forward pass selects a host-resident expert, the CPU computes it, or its weights are uploaded over PCIe to a GPU, either on demand or ahead of time by a predictor (Hwang et al., 2024; Xue et al., 2024; Song et al., 2024a). To hide the cost of host-resident experts, existing engines overlap CPU and GPU computation (Chen et al., 2025; Yang et al., 2026a) or cache frequently used experts on the GPUs (am17an, 2026; spiritbuun, 2026). These engines speed up decode on consumer hardware, but agent sessions expose two mismatches. First, **the wait after each tool call is a short prefill, too short for uploading experts to pay off**. Every agent framework turns a task into a session of requests, one after each tool output or user message: a held-out coding session in our traces sends 42 to 100 of them, and each request prefills only its newest tokens, a median of 249. Existing engines therefore keep these prefills away from the experts on their GPUs. The expert cache of llama.cpp (am17an, 2026), which ik_llama.cpp (ikawrakow, 2026) ports, serves only forward passes of up to 32 tokens, on the premise that larger ones use most experts of a layer, and the cache of the tuned fork buun (spiritbuun, 2026) serves only decode. In a short prefill, these engines therefore leave the experts in their caches unused, and KTransformers (Chen et al., 2025) computes on the CPU every selected expert outside the 58 per layer that it keeps on the GPUs. On our coding sessions, these engines keep the agent waiting a median of 2.4–2.8 s for the first token after each tool output (Figure 1a). Second, **existing engines make one phase of a turn fast at the expense of the other**. Their designs and evaluations center on decode speed and single requests (Chen et al., 2025; Yu et al., 2026; Yang et al., 2026a), but agent turns generate only a few hundred tokens after each prefill, so prefill takes 24–44% of the prefill and decode time of the engines that cache experts. Placing whole layers on the GPUs, as llama.cpp does by default, shortens these prefills but slows decode to 47.1 ms per output token, against 31.5 ms with its expert cache (Table 3). These mismatches raise the central question of this paper:

How can a CPU–GPU engine shorten the wait after each tool call without slowing decode or altering the model's outputs?

Our key observation is that a short prefill looks unsuited to the GPU only when it is counted in distinct experts rather than in token–expert routes. *Counted in experts*, a prefill of a few hundred tokens touches many experts of each layer, too many to upload for so few tokens, and caches skip it so as not to evict the experts that decode uses. *Counted in routes*, the same prefill is concentrated: expert routing in agent sessions is skewed and stable across sessions, and the 44% of experts that development sessions route to most often receive 84.0% of the routes of held-out sessions. A GPU that keeps these experts resident therefore serves most of every forward pass, prefill and decode alike, without uploading or evicting anything. Building on this observation, we present OSTINATO, a CPU–GPU MoE inference engine that computes the experts resident on the GPUs in every forward pass of an agent session. OSTINATO combines three techniques. First, **cross-layer expert residency** fills one pool per GPU, shared by its layers, with the experts that development sessions route to most often. A cost model calibrated inside the engine guides this choice and bounds its distance from the best static residency set by 0.15 ms per decode step. Second, **size-dependent prefill paths** serve a short prefill through a split path that computes its hot routes on the GPU and only its cold routes on the CPU, and a long prefill through an upload path that uploads each distinct cold expert once. Third, **a decode graph per GPU** computes the cold experts of a step on a resident CPU thread pool while the GPU computes the hot ones. All-zero slots (*zero slots*) take the place of cold experts, so the GPU's tensor shapes stay fixed however many selected experts are hot, so all layers of a GPU replay as one CUDA graph, and GPU memory that decode leaves idle holds a few more experts. Ultimately, our design rethinks short prefills in hybrid MoE inference: it serves them from the experts that the GPUs already hold rather than from the CPU alone, so the GPU memory that hybrid engines devote to decode works in every turn.

We implemented OSTINATO in llama.cpp and evaluated it against six CPU–GPU engines, llama.cpp in its default layer placement and with its expert cache, ik_llama.cpp, KTransformers, buun and FreeToken (Yang et al., 2026a), by replaying real coding (SWE) and customer-service (τ^2-bench) agent sessions with DeepSeek-V4-Flash and Qwen3-235B-A22B on a workstation with four RTX 4090D GPUs. On DeepSeek-V4-Flash, OSTINATO finishes every held-out

session and task sooner than every baseline and spends the least time of all engines in both prefill and decode (Figure 1). Against KTransformers and llama.cpp's expert cache, it cuts the median wait after a tool output by 47–55% and finishes sessions 1.4–2.5× sooner. Against llama.cpp's default layer placement, which prefills fastest among the baselines, it waits 22–24% less and decodes 1.8× faster, finishing sessions 1.7× sooner. On one GPU, it finishes the coding sessions 1.9× sooner than FreeToken. It also finishes every session sooner than the tuned llama.cpp forks ik_llama.cpp and buun, saving 16–36% of their total request time. On Qwen3-235B-A22B, the residency and prefill paths alone cut the total request time by 16.2% against buun.

This paper makes the following contributions:

- We design and implement OSTINATO, a CPU–GPU MoE inference engine for agents on local workstations. Guided by a cost model calibrated inside the engine, it computes the experts resident on its GPUs in every forward pass, from the short prefill after each tool output to each decode step, without changing what the model computes (Sections 3.1, 3.2 and 3.6).
- We design the execution paths that make this possible. Size-dependent prefill paths leave fewer than one of the six experts that a token selects per layer to the CPU in short prefills, and they nearly halve the prefill time in an ablation. A decode graph per GPU keeps its shapes fixed with zero slots and lends idle GPU memory to more experts; the graph alone cuts a decode step from 29.4 to 27.4 ms, and the full decode design reaches 25.6 ms (Sections 3.3, 3.4 and 4.5).
- We evaluate OSTINATO on real coding and customer-service agent sessions with DeepSeek-V4-Flash and Qwen3-235B-A22B against six CPU–GPU engines. OSTINATO finishes every held-out session and task sooner than every baseline, and its residency and prefill paths carry over to Qwen3-235B-A22B without changes to the model code (Sections 4.2, 4.3 and 4.6).

2. Background and Motivation

2.1. MoE Inference with Host-Resident Experts

An MoE layer (Shazeer et al., 2017; Lepikhin et al., 2021) replaces the feed-forward network of a transformer block with a router and E routed experts, of which each token uses k, plus shared experts that every token uses. Attention, router and shared-expert weights are used by every token, so they reside in GPU memory and run on the GPU. Routed experts are used sparsely, so the experts that do not fit, which we call *cold*, stay in host memory, while the experts in GPU memory are *hot*. A *forward pass* runs the model once over a batch of tokens. If it selects a cold expert, the engine either computes the expert on the CPU from host memory or uploads its weights over PCIe so that a GPU computes it. Which of the two is cheaper depends on how many tokens of the forward pass use the expert: an upload costs the same for one token or for many, whereas the CPU's cost grows with every token routed to the expert. An expert of DeepSeek-V4-Flash, for example, takes 13.4 MB in MXFP4 and 0.53 ms to upload over PCIe 4.0 on an idle bus, while our CPUs compute one token of it in about 0.18 ms. On an idle bus, an upload thus pays off only once about three tokens of the forward pass route to the expert.

Engines built on llama.cpp (The ggml authors, 2026) choose between the two by the size of the forward pass: below an *upload threshold*, cold experts run on the CPU, and from the threshold on, the selected experts are uploaded. Expert caches additionally keep frequently or recently used experts on the GPUs and admit new ones from decode steps and other small forward passes under a replacement policy (am17an, 2026; spiritbuun, 2026). They serve only such small forward passes: the cache of llama.cpp skips any forward pass of more than 32 tokens, which it expects to use most experts of a layer and to evict the experts used in decode. Compared with whole-layer placement, expert caches decode faster because more of each step's experts are already on the GPUs (31.5 against 47.1 ms per output token on our held-out SWE sessions). llama.cpp evaluates each forward pass as a computation graph and replays the GPU segments of decode steps as CUDA graphs (NVIDIA Corporation, 2026), which requires every step to keep the same operators and tensor shapes.

2.2. Agent Sessions on Hybrid Engines

Unlike single chat requests, agent sessions consist of many turns, each of which appends new messages or tool outputs to a long context, prefills them and generates a short response (Zhu et al., 2026a; Yuan et al., 2026). To examine the implications for hybrid engines, we replay software-engineering (SWE) sessions and τ^2-bench tasks (Section 4.1) with DeepSeek-V4-Flash on four RTX 4090D GPUs under the baselines of our evaluation. The results reveal that existing hybrid engines are ill-suited for agent sessions, primarily due to the following properties:

Short prefills on the CPU. Because each turn reuses the previous turn's KV cache, it prefills only its new tokens (a median of 249 per SWE turn and 118 per τ^2 turn), and three quarters of SWE turns prefill fewer than 512 tokens. Such forward passes are too large for the expert caches, which serve only forward passes of up to 32 tokens in llama.cpp and its port in ik_llama.cpp and only decode in buun. With its upload threshold of 512 tokens, llama.cpp's expert cache therefore computes all six experts that each token of a short prefill selects on the CPU, even those cached on the GPUs,

as buun does; KTransformers computes every selected expert outside the 58 per layer that it keeps on the GPUs on the CPU. These four engines take a median of 2.36–2.78 s on SWE and 1.80–2.11 s on τ^2-bench to return the first token after a tool output (Table 3).

One phase at a time. The engines trade the two phases of a turn against each other (Figure 6a and b). llama.cpp's default layer placement keeps whole MoE layers on the GPUs, so in a short prefill only the layers left in host memory compute their experts on the CPU, and its median wait after a tool output is 1.66 s on SWE and 1.25 s on τ^2-bench. In decode, however, each of these layers computes all its selected experts on the CPU at every step. Over our held-out sessions, the three expert caches spend 42–58% more time in prefill than layer placement but 16–44% less in decode.

Skewed, stable routing. Ranked by how often development sessions route to them, the top 44% of the experts receive 84.0% of the routes of held-out sessions, so the ranking carries over from one session to the next. Short prefills follow the same skew: although each touches many experts of a layer, the experts that OSTINATO keeps on the GPUs serve 84% of its routes (Section 2.3). A set of experts chosen offline could thus serve most routes of every forward pass.

These findings reveal a mismatch between existing hybrid engines and agent sessions: the GPU memory of each engine speeds up either the short prefills that the agent waits on or the decode steps that generate its responses, although one set of resident experts could serve both.

2.3. From CPU-Bound to GPU-Served Prefill

The above findings suggest serving short prefills from the experts already on the GPUs, which shortens the wait for the first token without giving up decode speed. We examine one held-out τ^2-bench turn whose times to first token lie close to the medians of such turns (Figure 2), and how the upload threshold changes the prefill time. Three insights follow from these measurements, and each becomes a design principle of OSTINATO.

(I) Resident experts can serve short prefills. When the agent of Figure 2 receives a tool output of 185 tokens, each layer routes 1,110 token–expert pairs, and llama.cpp's expert cache computes all of them on the CPU, returning the first token after 2.16 s. Uploading their cold experts would cost even more (II), but the routes concentrate on few experts: in short prefills of 128 to 511 new tokens, the residency set chosen on development sessions holds the experts of 84% of the routes. Only 0.92–0.95 of the six experts that a token selects per layer thus remain for the CPU, whereas llama.cpp's expert cache computes all six there. Since the GPU computes the hot experts from memory that it already holds, the CPU is left only the cold remainder, and OSTINATO returns

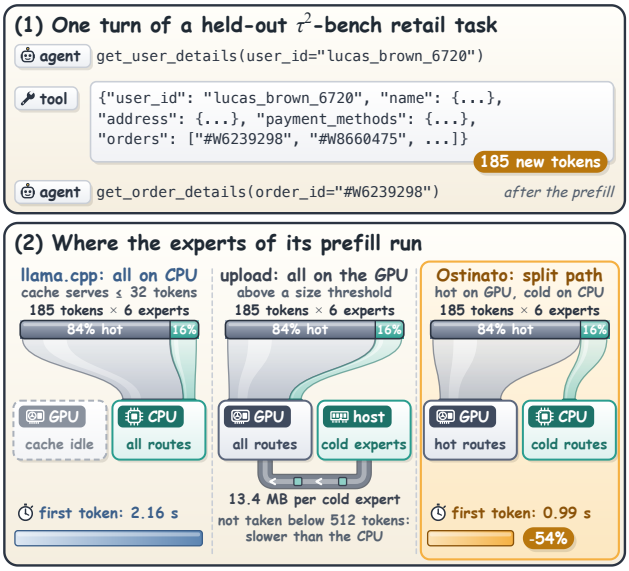

Figure 2. OSTINATO more than halves the wait after a typical tool output by computing the hot routes of its prefill on the GPU. (1) In a held-out τ^2-bench turn, the agent calls a tool, and the tool output adds 185 new tokens, which the engine prefills before the agent's next call. (2) Each bar splits the routes of these tokens into hot and cold (84% and 16% on average in short prefills of 128 to 511 new tokens), and the streams show where each part runs. llama.cpp's expert cache serves only forward passes of up to 32 tokens, so it computes all routes of this prefill on the CPU, including those whose experts it caches; an upload path moves every distinct cold expert over PCIe, which on our machine costs more than the CPU below 512 tokens; OSTINATO computes the hot routes on the GPU and only the cold routes on the CPU. The clocks give the measured time to first token of this turn, close to the medians over all 504 τ^2-bench turns that follow a tool output (2.11 s for llama.cpp's expert cache and 0.98 s for OSTINATO).

the first token of this turn after 0.99 s.

(II) Uploads pay off only for large forward passes. Since an upload moves an expert's 13.4 MB regardless of how few tokens use it, we swept the upload threshold on a development session, with buun as the engine. Its prefill time falls from 352.7 s with a threshold of 128 tokens and 329.2 s with 256 to 310.3 s with 512, and rises again to 323.6 s with 1,024. The forward passes that the lower thresholds move to the upload path are short prefills like the turn of Figure 2, so uploading their experts costs more than computing them on the CPU. A forward pass of 2,048 tokens, in contrast, touches 117 of a layer's 143 cold experts, and one upload per distinct expert is cheaper. A short prefill should therefore leave its cold experts to the CPU and a large one should upload them, which implies a single threshold on the size of a forward pass.

(III) Decode can hide the CPU's fixed cost. A decode step processes one token, so each layer selects only k experts, and a cold expert costs the CPU about 0.18 ms per route. Meanwhile, the GPU branch of a layer takes about 0.14 ms

regardless of how many hot experts it computes. The CPU can therefore compute a layer's cold experts while the GPU computes the hot ones, which hides nearly all of the CPU's fixed cost. Because the router's output is the only input that both branches need, a decode step can keep the same shapes regardless of the routing and replay all layers of a GPU as one CUDA graph, which hands the cold experts to the CPU without a host-driven launch and wait in every layer. Short prefills reuse the same index tables in their split path.

These observations motivate an execution design that uses the resident experts in every forward pass and sends each cold expert to the CPU or over PCIe by the size of the forward pass.

3. Design of OSTINATO

3.1. System Overview

OSTINATO works in two stages, offline residency planning and online execution, numbered as in Figure 3.

Offline residency planning. Executed once before deployment, this stage decides which experts stay on the GPUs (Figure 3a). OSTINATO ranks the experts of every layer by how often development sessions route to them (❶). A capacity test then fixes the number of expert slots on each GPU: the longest development request, with the workload's tool definitions, must complete while every GPU keeps a fixed margin free at its peak (❷). Finally, OSTINATO fills each GPU's slots with the highest-ranked experts of its layers, so the layers on a GPU share one expert pool and the number of slots that a layer receives follows its routing rather than a fixed quota. The resulting residency set is written to a file that the engine loads at start-up (❸).

Online execution. At start-up, OSTINATO copies the hot experts into the expert pool of each GPU and builds index tables that map every expert to a pool slot or to the CPU. Once the router selects the experts of a layer, the index tables split them into a *GPU branch* over the hot experts and a *CPU branch* over the cold experts (❹). In a decode step, a resident CPU thread pool computes the cold experts while the GPU computes the hot ones (❺), and a wait kernel joins the two branches before the GPU adds their outputs (❻). All layers on a GPU replay as one CUDA graph, and GPU memory that decode leaves unused holds a few more experts for later steps (Section 3.4). A prefill forward pass takes the split path or the upload path depending on its size (❼; Section 3.3). In every case, the layer's output is the routing-weighted sum of all k selected experts, as in the original model.

3.2. Cost Model and Expert Residency

Once the router has selected the experts of a decode step, each MoE layer ℓ divides them into its two branches. Let r_ℓ denote the number of *cold routes* of the layer, defined as the token–expert selections that name a cold expert. We denote the time of the GPU branch as G_ℓ, which does not depend on the number of hot routes because zero slots keep the GPU branch at k experts (Section 3.4). The CPU branch costs $C(r_\ell) = \gamma_0$ when $r_\ell = 0$ and $\alpha + \beta r_\ell$ otherwise, where α is its fixed cost and β its cost per cold route. Inside the engine on DeepSeek-V4-Flash, $\gamma_0 = 0.120$, $\alpha = 0.140$ and $\beta = 0.184$ ms, and $G_\ell = 0.135$–0.139 ms (Appendix B). By $\rho_{\ell e}$ we denote the number of routes to expert e of layer ℓ per decode step in a trace. For a residency set H_ℓ, the expected number of cold routes is $\mu_\ell = \sum_{e \notin H_\ell} \rho_{\ell e}$, and let q_ℓ represent the probability that the layer has any cold route. The two branches start when the routing result is ready and join before the next layer, so a layer costs $\max\{C, G_\ell\}$ when they overlap and $C + G_\ell$ when they run sequentially. We aim to minimize the expected time of a decode step, the sum of these costs over the layers.

Rule 1: keep the experts that remove the most cold routes. With $g_\ell = \max\{\gamma_0, G_\ell\}$, conditioning on r_ℓ gives, whenever $\beta \geq 0$ and $G_\ell \leq \alpha + \beta$,

$$\mathbb{E}\big[\max\{C(r_\ell), G_\ell\}\big] = g_\ell + \beta\mu_\ell + (\alpha - g_\ell)\, q_\ell, \qquad (1)$$

however the routes of a step correlate (Proposition B.1). On DeepSeek-V4-Flash, the residual $\alpha - g_\ell$ lies between 0.0013 and 0.0056 ms, about 3% of β, so the residency set should minimize the expected number of cold routes. Keeping the most frequently routed experts does so:

Proposition 3.1 (Frequency-ranked residency)**.** *Under the conditions of Equation (1), let the layers on each GPU share equal-size expert slots without per-layer limits, let all layers have the same β, and let $\alpha \geq g_\ell$ for every layer. Keeping the most frequently routed experts of each GPU minimizes the expected number of cold routes, and its expected decode-step time is within $\sum_\ell (\alpha - g_\ell)\, q_\ell$ of that of the best static residency set.*

On our development trace, this bound is 0.142 ms per decode step (Appendix B). Running the branches sequentially would instead reward completing whole layers, because every layer with any cold expert would add γ_0 to the step.

Capacity. Adding a slot never increases cold routes, so the expert pool fills all GPU memory left after the dense weights, the buffers of the largest forward pass and the KV cache of the longest development request. To leading order, a larger pool saves $\beta \sum_\ell \Delta\mu_\ell$ per decode step, which development traces give before the larger pool runs, and the full model adds at most 3% to it (Proposition B.2). Before we grew the pool from 4,864 to 5,702 hot experts, the model

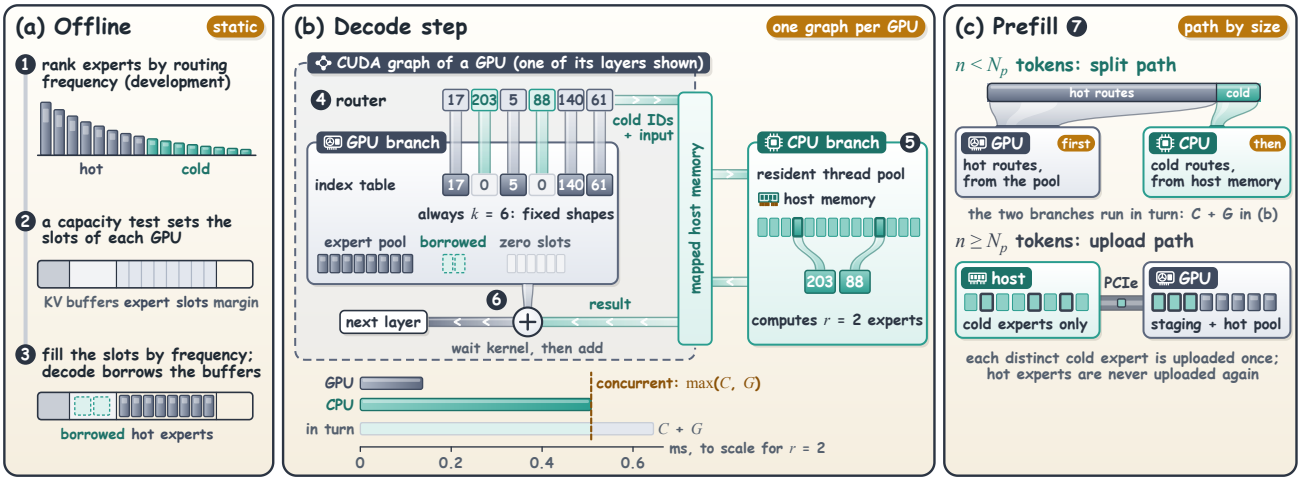

Figure 3. Overview of OSTINATO. The resident experts run on the GPU in every forward pass; only where the cold experts run changes. Slate: hot experts and GPU; teal: cold experts and CPU; charcoal: uploads over PCIe; black numbers: the steps of Section 3.1. (a) Offline, development sessions rank the experts (❶), a capacity test fixes the expert slots of each GPU (❷), and the highest-ranked experts fill them (❸); during decode, the buffers reserved for prefill hold borrowed experts. (b) In a decode step, the router selects $k = 6$ experts, two of them cold (❹). Inside the CUDA graph of the GPU (dashed), the index table passes all k IDs to the GPU branch and maps the cold ones to all-zero slots, so every step keeps the same shapes. The graph writes the cold IDs and the token's input into mapped host memory, a resident CPU thread pool outside the graph computes only the $r = 2$ cold experts (❺), and a wait kernel joins the two branches before the next layer (❻). The timeline is drawn to measured scale for $r = 2$. (c) A prefill forward pass of fewer than $N_p = 512$ tokens splits its routes between the two branches, which run in turn; a larger one uploads each of its distinct cold experts once (❼).

predicted a saving of 2.24 ms per step, and held-out sessions measured 2.25 ms.

Rule 2: compute a decode step's cold experts on the CPU. Uploading a cold expert for the current token exposes its transfer (0.53 ms on an idle bus and up to 1.20 ms while the CPU reads host memory). The CPU branch, in contrast, runs beside the GPU branch and exposes about β per cold route. A decode step therefore computes its cold experts on the CPU. Uploading an expert for later steps is a separate decision, which pays when the CPU time that its later routes save exceeds the transfer that the GPU cannot hide. OSTINATO makes it only in GPU memory that decode would leave unused (Section 3.4), and the residency set itself stays fixed during the run.

Rule 3: switch prefill paths once, by the size of the forward pass. An n-token prefill forward pass with $r(n)$ cold routes can take one of two paths. The *split path* computes the hot routes on the GPU and the cold routes on the CPU, and the *upload path* uploads its $u(n)$ distinct cold experts once and computes everything on the GPU. The CPU's cost grows with the cold routes, whereas the upload cost grows with the distinct cold experts, which the experts of a layer bound:

Proposition 3.2 (One switch between the prefill paths)**.** *Let the routes of a prefill forward pass be stationary over its tokens, let $\bar{u}(n)$ be the expected number of distinct cold experts that its first n tokens select, and let the expected cost difference between the split and the upload path be $\phi_0 + \phi_1 n - \phi_2\,\bar{u}(n)$, with $\phi_0 < 0$, $\phi_1 > 0$ and $\phi_2 > 0$. Then there is an n^* such that the split path is cheaper for every $n < n^*$ and the upload path for every $n > n^*$.*

A single prefill threshold N_p therefore suffices (Section 3.3; proof in Appendix B).

3.3. Size-Dependent Prefill Paths

Split path. A prefill forward pass with fewer than N_p tokens takes the split path. As in decode, the CPU computes the cold routes from host memory and the GPU computes the hot routes from the pool, but the two branches run one after the other. The GPU thus serves the large majority of the routes of the forward pass from experts that it already holds, and nothing crosses the PCIe link.

Upload path. A forward pass with N_p or more tokens takes the upload path, whose cold branch is a GPU expert operator over a compact host tensor that holds only the layer's cold experts. OSTINATO translates each cold route to its expert in this tensor and each hot route to a zero slot (an all-zero expert that contributes nothing). The upload therefore copies only the distinct cold experts of the forward pass into a staging buffer. The GPU then computes the uploaded experts from the staging buffer and the hot experts from the pool, and hot experts are never uploaded again.

Threshold. We set $N_p = 512$ tokens, where the CPU and upload paths of llama.cpp cross on our machine (between 512 and 1,024 tokens) and where buun prefills a

development session fastest (Section 2.3). A single path for all forward passes would be slow for one of the two kinds: uploads waste PCIe time on small forward passes, whose cold experts serve few tokens, whereas the CPU is far slower than one upload of the distinct cold experts of a large forward pass.

3.4. Decode in One Graph per GPU

Fixed shapes through zero slots. A CUDA graph replays fixed operators on fixed shapes, but the number of hot experts that a token selects changes from step to step. Each GPU's pool therefore reserves one zero slot for each of the k positions, and the GPU index table maps every cold expert to the zero slot of its position. The GPU branch thus always computes k distinct experts, and the graph keeps the same operators and shapes in every decode step, regardless of the routing. OSTINATO builds on this fixed shape to capture all layers on a GPU as one CUDA graph. In each layer, the graph publishes the routing result and whether any selected expert is cold. For a layer with cold experts, it writes the token's activations and the cold expert IDs into a mapped host buffer, from which a resident CPU thread pool computes the CPU branch and writes the result back. A wait kernel in the graph blocks only until that result arrives, after which the GPU adds the two branches, and a layer whose experts are all hot skips the CPU entirely. The GPU branch and the shared expert run while the CPU computes, so a step pays only for the part of the CPU branch that the GPU work does not hide, as Rule 1 assumes. Zero slots occur only in layers with a cold route, where the CPU branch takes at least $\alpha + \beta = 0.324$ ms, more than twice G_ℓ, so the GPU computes them within its wait for the CPU and they add no time to a decode step.

Borrowed memory. The memory that the capacity test reserves for the largest prefill forward pass lies unused while sessions decode, and OSTINATO lends it to the cold experts that decode keeps computing on the CPU. Every CPU use raises an expert's score, which decays over decode steps. A background thread uploads the highest-scored experts into free slots of the GPU that holds their layer, and once these slots are full, a newcomer replaces the lowest-scored borrowed expert only when it scores clearly higher. An uploaded expert joins the GPU branch at a later token boundary, where the index tables of both branches switch together. A prefill forward pass returns all borrowed memory before it runs, so a borrowed expert serves only the decode of one request. Because the uploads overlap with the GPU's work, the GPU waits for them about 0.01 ms per decode step, and the decode step becomes 1.4 ms shorter on SWE and 0.4 ms shorter on τ^2-bench (Section 4.5).

Alternatives. Dispatching each cold expert from the host (Kamahori et al., 2025) adds a host operation per expert and prevents graph replay. SeqMoE keeps a fixed expert pool with lookup tables so that its decode loop can be captured as a graph (Wang et al., 2026b). OSTINATO adds a zero slot per position, a pool shared across layers, the prefill paths and the CPU branch inside the graph.

3.5. Running Other MoE Models

The residency planner, the capacity test, the index tables and both prefill paths work on the expert tensors of each layer, not on model-specific code. They therefore run Qwen3-235B-A22B, whose 94 MoE layers each route a token to 8 of 128 experts, without changes to the model code. Its Q4_K_M experts store their projections in two quantization types, so the expert pool groups resident experts by their actual size in bytes. On Qwen3-235B-A22B, decode runs the two branches with llama.cpp's per-segment CUDA graphs, so its results isolate the gains of the residency planner and the prefill paths on a second architecture.

3.6. Implementation

We prototype OSTINATO on top of llama.cpp (The ggml authors, 2026) with CUDA 12.6 and keep llama.cpp's native expert operators for both branches. The two branches, the index tables, the zero slots, the decode graph and the borrowed memory add about 4.3K lines of C++ and CUDA to llama.cpp. The upload path builds on llama.cpp's selective upload, which copies only the experts that the routes of a forward pass reference. The residency planner, the capacity test and the selection of the borrowing parameters run offline. OSTINATO pre-allocates the expert pool and the staging buffer of the upload path before it counts slots, so the pool never competes with prefill for memory at run time. On our machine, the borrowed memory holds 544 expert slots across the four GPUs, and the background thread copies experts in chunks that alternate between the two PCIe root complexes. To keep a whole decode step replayable, the mapped buffers between each GPU and the CPU thread pool are allocated once at start-up and reused by every step.

4. Evaluation

We evaluate OSTINATO to answer five questions: (1) how much total request time it saves against existing CPU–GPU engines (Section 4.2), (2) how much it shortens the wait after tool outputs and user messages (Section 4.3), (3) how the wait changes with the number of new tokens (Section 4.4), (4) what each component contributes (Section 4.5), and (5) whether the general residency and prefill paths carry over to Qwen3-235B-A22B (Section 4.6).

Table 1. Evaluated models. Date: release month; Params: total / active; Experts (k): MoE layers × routed experts in each layer, and in parentheses the number that each token selects. DeepSeek-V4-Flash also has one shared expert per layer. [a]Without its speculative-decoding module.

Model	Date	Params	Experts (k)	Format
DeepSeek-V4-Flash	2026-07	284B/13B[a]	43×256 (6)	MXFP4
Qwen3-235B-A22B	2025-07	235B/22B	94×128 (8)	Q4_K_M

Table 2. Replayed workloads. Outputs: generated tokens; New: median number of tokens prefilled per turn after reuse of the previous turn's KV cache. [a]The first 30 turns of two development sessions, used for ablations.

Set	Sessions	Turns	Outputs	New
Software engineering (SWE)				
development	6	400	105,879	258
screening[a]	2	60	9,842	335
held-out	13	893	215,341	249
τ^2-bench				
development	24	335	32,931	135
held-out	60	916	83,687	118

4.1. Experimental Setup

Models. We evaluate OSTINATO on two publicly available MoE models from two families. Specifically, we include DeepSeek-V4-Flash (DeepSeek-AI, 2026b) (V4), on which we run the full method, and Qwen3-235B-A22B (Qwen Team, 2025) (Qwen3), on which we run the general residency and prefill paths (Section 3.5). The two models differ in how many experts they have, how wide each expert is and how many layers hold them. Table 1 lists their main parameters. We serve the routed experts of V4 in its native MXFP4 format and those of Qwen3 in llama.cpp's Q4_K_M format, both on four GPUs.

Workloads. To evaluate performance, we replay agent trajectories from two public collections (Table 2): OpenHands (Wang et al., 2025) sessions that resolve SWE-rebench issues (Trofimova et al., 2025; Badertdinov et al., 2025) and τ^2-bench conversations (AgentSuite, 2026; Barres et al., 2026) in the airline, retail and telecom domains, each with its recorded tool outputs. All experiments serve one session at a time, representing the typical local deployment of an agent. On V4, we replay all 13 held-out SWE sessions and 60 held-out τ^2-bench tasks; on Qwen3, we replay the same 60 tasks, whose trajectories Qwen3 itself generated. Development sessions choose the residency sets and thresholds, and held-out sessions are used only for evaluation. Every engine is forced to emit the recorded tokens, so all systems receive and emit the same tokens (Pan et al., 2026). To isolate the effect of input length, we also prefill 128 to 1,024 new tokens after eight development prefixes and generate 128 tokens (Section 4.4).

Hardware. We adopt a workstation with four RTX 4090D GPUs (24 GB each) on PCIe 4.0, two Intel Xeon Platinum 8358 CPUs (64 cores) and 512 GB of DDR4 memory as our testbed. All systems run on this machine, and its CPUs read host memory at 86–88 GB/s.

Baselines. We compare OSTINATO with six CPU–GPU MoE inference systems, each a distinct way of placing experts. All of them run on the same machine and pass the same capacity test, in which the longest development request must complete (Appendix C). (1) *llama.cpp (layers)* (The ggml authors, 2026) maps each MoE layer as a whole to a GPU or to the CPU, and we let its automatic fit choose the layers. (2) *llama.cpp expert cache* (am17an, 2026) keeps experts in a GPU cache shared by the layers of each device and uploads missing experts on demand; we use the largest cache budget that passes the capacity test. (3) *ik_llama.cpp* (ikawrakow, 2026) is a llama.cpp fork with its own CPU kernels and a GPU cache for host-resident experts (SamuelOliveirads, 2026), with the cache budget of 69,632 MiB chosen on development sessions. (4) *buun* (spiritbuun, 2026) is a llama.cpp fork with a dynamic expert cache that admits experts by decaying access counts and is warmed with a routing profile of the development sessions; we use the upload threshold of 512 tokens chosen on a development session. (5) *KTransformers* (Chen et al., 2025; kvcache-ai, 2026) keeps a fixed number of experts per layer on the GPUs and computes the others on the CPU. We use its default uniform placement and the fastest of seven configurations on a development session, which keeps 58 experts per layer on the GPUs, and we replay it with a separate lean client because it serves a different HTTP interface. (6) *FreeToken* (Yang et al., 2026a), an edge-native MoE engine, runs on one GPU, so we compare it with a one-GPU configuration of OSTINATO with the same 32 CPU cores.

Evaluation Metrics. We use the total request time (the sum of the wall-clock times of all requests in a session) as our primary metric, and we also report the time to first token (TTFT) and the time per output token (TPOT). TTFT, a standard latency metric for LLM serving, runs from the arrival of a request to its first output token. We report it for every turn that follows a tool output, measured at the client from sending the request that carries the tool output to receiving the first new token, and report the τ^2-bench turns that follow a user message separately. TPOT is the interval between the first and the last output token divided by the number of output tokens minus one. For each reduction of the total request time, Table 3 gives a 95% confidence interval from a bootstrap over sessions or tasks. Every engine returned the recorded tokens in every turn of every run, and under the same expert placement and routing, the decode path of OSTINATO is bitwise identical to a synchronous reference (Appendix A).

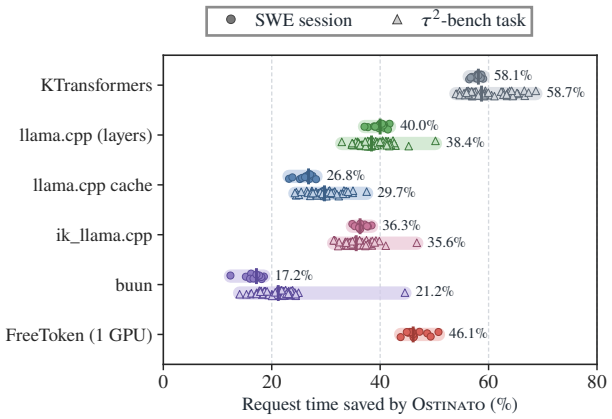

Figure 4. OSTINATO finishes every held-out SWE session and τ^2-bench task sooner than every baseline on DeepSeek-V4-Flash, saving at least 12.4% of the request time in each. Points: request time that OSTINATO saves in each session or task against each baseline (FreeToken: one GPU, SWE only); labels: median saving.

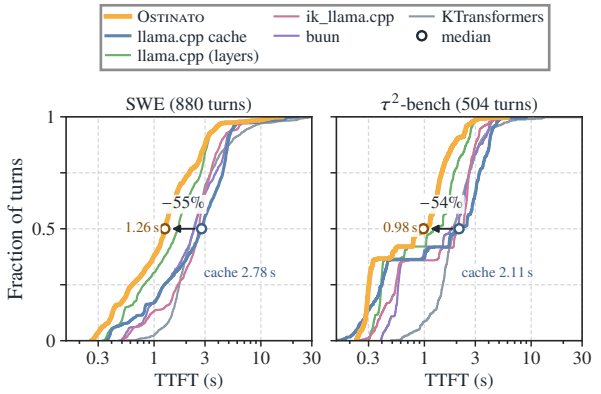

Figure 5. OSTINATO has the shortest median wait after a tool output on both workloads, 55% and 54% below that of llama.cpp's expert cache. Time to first token (TTFT) of every turn that follows a tool output on DeepSeek-V4-Flash: all 880 such turns of the held-out SWE sessions and 504 of the held-out τ^2-bench tasks. Curves further left mean shorter waits; circles: medians of llama.cpp's expert cache and OSTINATO; arrows: the change of the median. The axis ends at 30 s, beyond which fewer than 1% of any engine's turns lie.

4.2. Main Results

Table 3 compares the total request time, the time to first token after a tool output and the time per output token of OSTINATO with each baseline on the held-out sessions of both workloads, and Figure 1b shows the same totals as speedups. OSTINATO finishes every SWE session and every τ^2-bench task sooner than every baseline, saving at least 12.4% of the request time in each of them (Figure 4). Against KTransformers, it saves 58.0% and 60.4% of the total request time on the two workloads, finishing sessions 2.4× and 2.5× sooner. KTransformers keeps 58 experts per layer on the GPUs, fewer than half of the 133 that OSTINATO keeps on average, and its prefill kernels target Intel AMX, which our CPUs lack. Against llama.cpp, OSTINATO saves 40.1% and 39.6% in its default layer placement and 26.1% and 29.7% with its expert cache. On one GPU with the same 32 CPU cores, it saves 46.8% against FreeToken, finishing sessions 1.88× sooner (Figure 1b; settings in Appendix C). Against the tuned llama.cpp forks, it saves 36.5% and 36.1% with ik_llama.cpp and 16.5% and 20.4% with buun, the baseline with the shortest total request time.

Figure 6a and b show where the savings come from. Among the llama.cpp-based baselines, layer placement prefills fastest but decodes slowest, while the expert caches decode faster but spend 42–58% more time in prefill. OSTINATO is shorter than all of them in both phases. Against layer placement, it shortens prefill by 12.9% on SWE and 16.8% on τ^2-bench and decode by 44.9% and 46.6%. Against the fastest-decoding baseline, buun, it shortens prefill by 40.1% and 41.6% and still shortens decode by 6.4% and 4.1%. Decode stays fastest as contexts grow: in each quartile of the SWE turns by context length, OSTINATO takes 25.6–26.5 ms per output token, against 26.1–28.3 ms for buun and 29.8–32.4 ms for llama.cpp's expert cache (Figure 6d). The prefill paths of Section 3.3 and the decode graph of Section 3.4 produce these gains, and Section 4.5 separates their contributions.

4.3. Waiting after Tool Calls

We evaluate the wait after tool outputs by comparing the time to first token of OSTINATO with that of the baselines on every turn that follows a tool output. Figure 5 presents the distribution of these times for every system over the 880 SWE turns and the 504 τ^2-bench turns, and Table 3 and Figure 6c give the medians. OSTINATO has the shortest median on both workloads, 1.26 s on SWE and 0.98 s on τ^2-bench. This is 46.6% and 48.0% shorter than the median of KTransformers, 54.7% and 53.7% shorter than that of llama.cpp's expert cache and 24.2% and 22.0% shorter than that of its layer placement. Against the forks, the medians fall by 51.7% and 53.0% for ik_llama.cpp and by 46.7% and 45.5% for buun. Every baseline serves short prefills from the CPU or by uploading experts, which leaves their medians on SWE between 1.66 s and 2.78 s. OSTINATO also shortens the τ^2-bench turns that follow a user message, from 0.47 s for llama.cpp's layer placement and 0.52 s with its expert cache to 0.37 s (Table 7). On the first turn of a session, which prefills more than 1,400 new tokens and takes the upload path, OSTINATO is faster than every expert cache and KTransformers and within 0.08–0.12 s of llama.cpp's layer placement (4.70 against 4.57 s on SWE and 3.90 against 3.82 s on τ^2-bench; Table 7). On the other turns, the split path drives the reductions, because it serves their short prefills from resident experts (Section 4.4).

Table 3. OSTINATO has the shortest total request time, time to first token and time per output token on both workloads. Held-out results on DeepSeek-V4-Flash. Time: total request time of the 13 SWE sessions or the 60 τ^2-bench tasks; Saved: share of that time that OSTINATO saves, with its 95% confidence interval below; Won: sessions or tasks that OSTINATO finishes sooner; TTFT: median time to first token after a tool output; TPOT: median time per output token. Best values in bold.

	SWE (13 sessions)					τ^2-bench (60 tasks)				
Engine	Time (s)	Saved	Won	TTFT (s)	TPOT (ms)	Time (s)	Saved	Won	TTFT (s)	TPOT (ms)
KTransformers	16,950.1	58.0% 57.6–58.4	13/13	2.36	62.8	7,739.6	60.4% 59.9–61.0	60/60	1.88	61.6
llama.cpp (layers)	11,880.4	40.1% 39.3–40.7	13/13	1.66	47.1	5,073.9	39.6% 38.9–40.3	60/60	1.25	46.7
llama.cpp expert cache	9,638.7	26.1% 25.4–26.9	13/13	2.78	31.5	4,357.2	29.7% 28.9–30.6	60/60	2.11	30.0
ik_llama.cpp	11,207.8	36.5% 36.0–37.0	13/13	2.61	39.6	4,795.4	36.1% 35.5–36.9	60/60	2.08	36.5
buun (llama.cpp fork)	8,523.9	16.5% 15.5–17.4	13/13	2.37	27.3	3,846.2	20.4% 19.6–21.2	60/60	1.80	26.0
OSTINATO	**7,119.0**	–	–	**1.26**	**26.0**	**3,062.8**	–	–	**0.98**	**25.3**
One GPU and 32 CPU cores, SWE sessions (mean of two runs per session)										
FreeToken	27,669.3	46.8% 45.8–47.8	13/13	–	–					
OSTINATO	**14,714.6**	–	–	–	–					

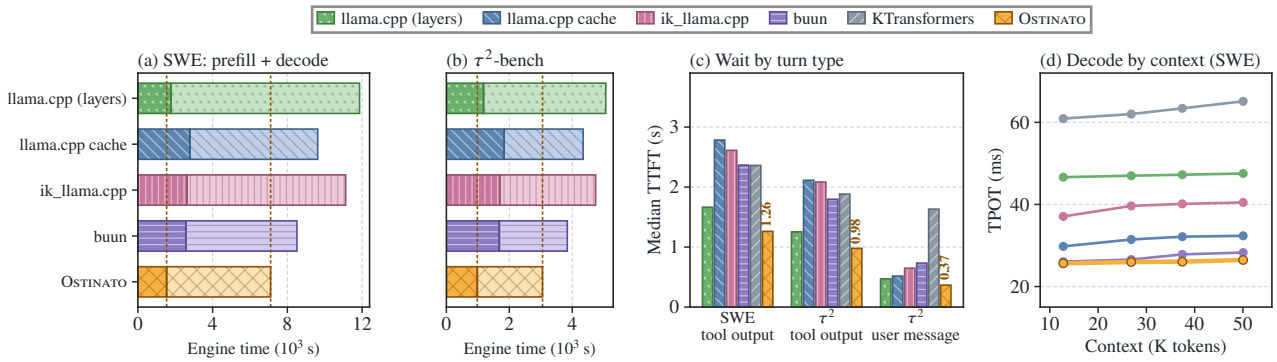

Figure 6. OSTINATO spends less time than every llama.cpp-based baseline in both prefill and decode, has the shortest median wait after tool outputs and user messages, and decodes fastest at every context length. (a, b) Prefill (dark) and decode (light) time of all held-out sessions on DeepSeek-V4-Flash as reported by each engine; dashed lines: prefill time and total of OSTINATO. (c) Median time to first token after a tool output and after a user message (first turns in Table 7). (d) Median time per output token of the held-out SWE turns in quartiles of context length, plotted at the median context of each quartile; per turn, the time from the first token to the end of the request divided by the output tokens minus one.

4.4. Input Length

Figure 7a groups the 1,384 held-out turns that follow a tool output by their number of new tokens. Below 512 new tokens, where OSTINATO takes the split path, it has the shortest median time to first token in every group, 22.0–25.2% shorter than that of llama.cpp's layer placement, the fastest baseline in these groups, and 31.0–56.1% shorter than those of the expert caches. With 512 to 1,023 new tokens, where both OSTINATO and layer placement upload weights to the GPUs, its lead over layer placement is 7.9%. With 1,024 or more new tokens (85 turns), the two wait about as long (3.87 against 3.78 s), and OSTINATO still waits 29.3–37.6% less than the expert caches.

To separate input length from context length, we also prefill 128, 343, 511, 512 and 1,024 new tokens after eight development prefixes of 7.4K–39.3K tokens and generate 128 tokens (Figure 7b). We compare OSTINATO with buun in this sweep because buun switches from computing experts on the CPU to uploading them at the same threshold of 512 tokens. With up to 512 new tokens, OSTINATO takes the split path and reduces the median time to first token by 43.7–45.9%, from 1.63 to 0.92 s with 128 new tokens and from 3.98 to 2.16 s with 512. Over this range, its time to first token grows by 3.2 ms per new token, about half of the 6.0 ms of buun, because OSTINATO computes on the CPU only 0.92–0.95 of the six experts that a token selects per layer, whereas buun and llama.cpp's expert cache compute

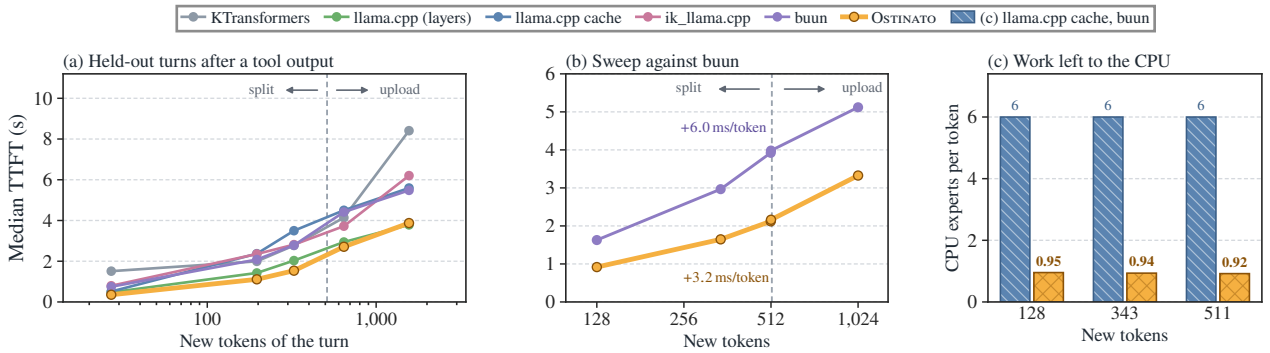

Figure 7. Below 512 new tokens, where OSTINATO takes the split path, it waits least in every group, its wait grows about half as fast with input length as that of buun, and its CPU computes 6.3–6.5× fewer experts per token. (a) Median time to first token of the 1,384 held-out turns that follow a tool output on DeepSeek-V4-Flash (880 SWE and 504 τ^2-bench turns), grouped by new tokens (fewer than 128, 128–255, 256–511, 512–1,023 and at least 1,024) and plotted at the median length of each group. (b) Prefilling 128 to 1,024 new tokens after eight development prefixes, against buun, which switches from computing experts on the CPU to uploading them at the same 512-token threshold; labels: least-squares slope on the split path. (c) Experts that the CPU computes per token and layer in prefills of 128, 343 and 511 new tokens after the same prefixes: all six for llama.cpp's expert cache and buun, whose caches skip forward passes of more than 32 tokens, and 0.92–0.95 for OSTINATO.

all six there (Figure 7c). With 1,024 new tokens, both engines take their upload paths, and OSTINATO reduces the median by 35.1% (from 5.12 to 3.33 s); it uploads only the cold experts that the batch selects (3,243 at the median). Because llama.cpp's context checkpoints prefill the last four tokens of a request as a separate batch, the first request that takes the upload path carries 516 new tokens.

4.5. Ablation

To separate what each part of OSTINATO contributes, we add its components one at a time: first the resident experts with their prefill paths, then the decode graph with its borrowed memory. Figure 8 summarizes their impact on prefill, decode and total time. Its prefill panels use the 60-turn screening set and the 893 held-out SWE turns of DeepSeek-V4-Flash, with llama.cpp's layer placement and expert cache as references; its decode panel uses two development prefixes per workload, with buun, the fastest-decoding baseline, as the reference.

We first keep the experts resident on the GPUs, with overlapped branches, but let prefill take llama.cpp's native path (Figure 8a). By serving decode from resident experts, this configuration already decodes as fast as the full system (295.2 against 295.8 s on the screening set). Its prefill, however, takes 284.8 s, longer than that of both references, because the native path ignores the resident experts. Next, we enable the size-dependent prefill paths. This reduces prefill time by 47%, from 284.8 to 150.8 s, and the total time from 581.0 to 447.7 s, with decode unchanged. On the held-out sessions (Figure 8b), the same components shorten the replay step by step. Resident experts cut the 12,084.9 s of layer placement to 9,753.8 s, overlapped branches to 8,679.3 s, and a larger pool with the prefill paths to 8,116.6 s, against 9,651.1 s for llama.cpp's expert cache. Finally, the decode panel (Figure 8c) adds the decode graph and borrowed memory to the two-branch path. The decode graph saves about 1.9 ms per step on both workloads, avoiding repeated thread-affinity calls saves another 0.2–0.4 ms, and borrowed memory saves 1.4 ms on SWE and 0.4 ms on τ^2-bench. OSTINATO thus decodes a step in 25.6 and 24.8 ms against 28.7 and 27.9 ms for buun. With this decode path, the held-out SWE sessions decode in 5,566.3 s instead of the 6,588.3 s of the last row of Figure 8b, and their total request time falls from 8,116.6 s to the 7,119.0 s of Table 3. The prefill paths and the decode graph are thus complementary: the former shortens prefill, the latter shortens decode, and together they put OSTINATO ahead of every baseline in both phases (Figure 6a).

Table 4. On Qwen3-235B-A22B, the general residency and prefill paths cut the median time to first token by 39–58% and the total request time by 16.2% against buun. Median time to first token by turn type on the 60 held-out τ^2-bench tasks, buun → OSTINATO, and the total request time.

Turn type	Turns	TTFT (s)	Change
after a tool output	504	2.75→1.67	−39.4%
after a user message	352	1.28→0.54	−57.8%
total request time	916		−16.2%

4.6. Qwen3-235B-A22B on the General Paths

We evaluate the general paths on Qwen3 against buun, the baseline with the shortest total request time on DeepSeek-V4-Flash, on the 60 held-out τ^2-bench tasks; on this model,

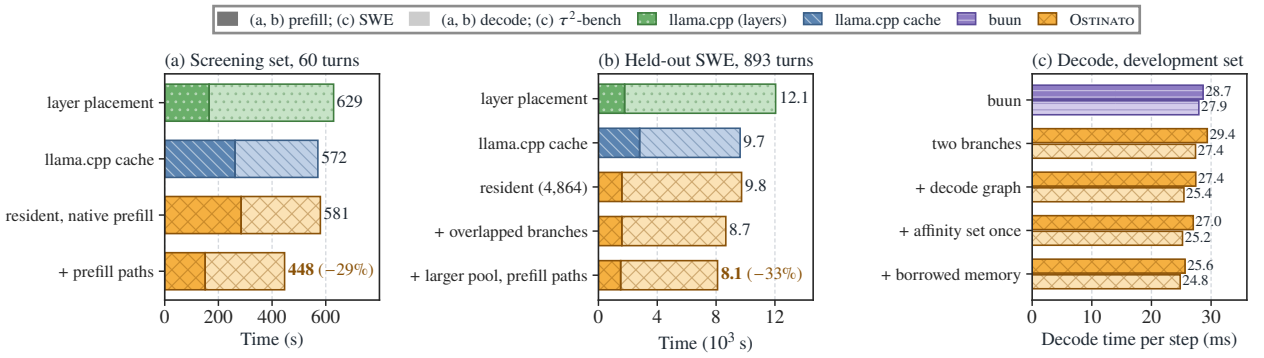

Figure 8. The prefill paths nearly halve the prefill time, and the decode design cuts a decode step from 29.4 to 25.6 ms on SWE, below buun's 28.7 ms. Ablation on DeepSeek-V4-Flash; below the references, each row adds one component to the row above. (a) Screening set (60 turns) and (b) held-out SWE sessions (893 turns): prefill and decode time of each configuration, labeled with its total time. Panels (a) and (b) decode with the two branches and llama.cpp's per-segment CUDA graphs, without the decode graph and borrowed memory of (c), with 5,702 resident experts in (a) and 4,864 growing to 5,702 in the last row of (b); with the full decode path, the sessions of (b) take the 7,119.0 s of Table 3. (c) Time per decode step on two development prefixes per workload, which isolates the decode components, with buun as the reference; the prefill of each workload changes by less than 0.2 s across the rows. Appendix D lists all numbers.

OSTINATO decodes without the decode graph and borrowed memory. OSTINATO finishes 59 of the 60 tasks sooner and reduces the total request time by 16.2% (Table 4). The gain comes from prefill, which takes 41.2% less time, while decode, still on llama.cpp's per-segment CUDA graphs, stays within 3.1% of that of buun. The median time to first token falls by 39.4% after tool outputs and by 57.8% after user messages (Table 4). Table 4 leaves out the first turn of each task, where buun delays its first token by a median of 1.86 s beyond its prompt time (Appendix C). Without any model-specific code, the residency set and the prefill paths thus shorten the wait after tool outputs on a second model family with 94 MoE layers and experts of two sizes.

5. Related Work

MoE routing and expert locality. Recent open-weight MoE models route each token to a few of many fine-grained experts, such as 6 of 256 per layer in DeepSeek-V4-Flash and 8 of 128 in Qwen3-235B-A22B (DeepSeek-AI, 2026b; Yang et al., 2025). To make expert accesses local, Oracle-MoE trains a router so that consecutive tokens activate consistent experts (Zhou et al., 2025a), ReMoE fine-tunes only the gate of a trained model (Zhu et al., 2026b), OEA reroutes tokens toward experts that the batch already loads (Oncescu et al., 2026), and FloE compresses experts (Zhou et al., 2025b). OSTINATO addresses the same cost without touching the model: it exploits the skew that the unchanged router already shows across agent sessions, so it computes the original model and needs no accuracy evaluation.

MoE inference with host-resident experts. MoE-Infinity, HOBBIT and Pre-gated MoE cache or prefetch experts on the GPU (Xue et al., 2024; Tang et al., 2024; Hwang et al., 2024), and ZipMoE compresses expert weights losslessly to cut I/O on edge devices (Yang et al., 2026b). Fiddler, MoE-Lightning, KTransformers, HybriMoE and FreeToken split expert computation between the CPU and the GPU (Kamahori et al., 2025; Cao et al., 2025; Chen et al., 2025; Zhong et al., 2025; Yang et al., 2026a). llama.cpp offers whole-layer placement and an expert cache for host-resident experts (The ggml authors, 2026; am17an, 2026), and its fork buun adds a dynamic cache with decaying access counts (spiritbuun, 2026). OSTINATO differs from these engines in using its resident experts in every forward pass: its split path serves short prefills from GPU-resident experts, which existing caches leave to the CPU or to uploads. KTransformers places shared experts on the GPU and routed experts on the CPU, and it fits the whole decode path of a token into one CUDA graph by issuing the CPU's work through host callbacks (Chen et al., 2025). The decode graph of OSTINATO also computes routed experts on the GPUs, whose selection changes with every token: zero slots keep their shapes fixed, and a wait kernel on the device joins the CPU branch. For dense LLMs, PowerInfer keeps frequently activated neurons on the GPU and computes the rest on the CPU (Song et al., 2024b); OSTINATO answers the corresponding question for MoE models in both phases. Lossless compression (Yang et al., 2026b) is complementary to OSTINATO: it lowers the cost of each transfer or host read, whereas OSTINATO lowers how many expert computations the CPU and the PCIe link must serve.

Serving LLMs with tool calls. InferCept decides for each paused request whether to keep, swap or recompute its KV cache (Abhyankar et al., 2024), and AugServe orders requests and adapts batch budgets to the states that tool calls create (Wang et al., 2026a). Cake computes part of a cached

prefix on the GPU while loading the rest from storage (Jin et al., 2025), prefix caching avoids recomputing earlier turns (Kwon et al., 2023; Zheng et al., 2024; Gim et al., 2024), and DistServe places prefill and decode on different GPUs (Zhong et al., 2024). These systems reduce the work that a returning request needs, whereas OSTINATO reduces the time that this work takes on a hybrid machine, and it reuses the KV cache of earlier turns as they do.

6. Conclusion

We presented OSTINATO, a CPU–GPU MoE inference engine for agents on local workstations. Exploiting the skewed and stable routing of agent sessions, OSTINATO keeps the experts that these sessions route to most often on the GPUs and computes them there in every forward pass. Short prefills split their routes between the GPU and the CPU, long prefills upload each cold expert once, and decode steps run as one CUDA graph per GPU, with a cost model calibrated inside the engine guiding these choices. On replayed coding and customer-service agent sessions, OSTINATO spends less time in both prefill and decode than the four-GPU baselines on DeepSeek-V4-Flash, and its residency and prefill paths alone shorten the sessions of Qwen3-235B-A22B, all without changing what the model computes.

References

Abhyankar, R., He, Z., Srivatsa, V., Zhang, H., and Zhang, Y. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pp. 81–95. PMLR, 2024. URL https://proceedings.mlr.press/v235/abhyankar24a.html.

AgentSuite. tau2-bench agent trajectories, 2026. URL https://huggingface.co/datasets/AgentSuite/tau2-bench-trajectories. Hugging Face dataset. Accessed 2026-10-30.

am17an. llama : add a GPU cache for MoE experts kept in host memory, 2026. URL https://github.com/ggml-org/llama.cpp/pull/29887. llama.cpp pull request #29887. Accessed 2026-10-30.

Badertdinov, I., Golubev, A., Nekrashevich, M., Shevtsov, A., Karasik, S., Andriushchenko, A., Trofimova, M., Litvintseva, D., and Yangel, B. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. In *Advances in Neural Information Processing Systems 38 (NeurIPS), Datasets and Benchmarks Track*, 2025. doi: 10.52202/085713-0788. URL https://papers.nips.cc/paper_files/paper/2025/hash/21bec6ace947b1b58967b945c8ac0f10-Abstract-Datasets_and_Benchmarks_Track.html.

Barres, V., Dong, H., Ray, S., Si, X., and Narasimhan, K. R. τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pp. 6736–6782. PMLR, 2026. URL https://proceedings.mlr.press/v306/barres26a.html.

Cao, S., Liu, S., Griggs, T., Schafhalter, P., Liu, X., Sheng, Y., Gonzalez, J. E., Zaharia, M., and Stoica, I. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS)*, pp. 715–730. ACM, 2025. doi: 10.1145/3669940.3707267. URL https://arxiv.org/abs/2411.11217.

Chen, H., Xie, W., Zhang, B., Tang, J., Wang, J., Dong, J., Chen, S., Yuan, Z., Lin, C., Qiu, C., Zhu, Y., Ou, Q., Liao, J., Chen, X., Ai, Z., Wu, Y., and Zhang, M. KTransformers: Unleashing the Full Potential of CPU/GPU Hybrid Inference for MoE Models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP)*, pp. 1014–1029. ACM, 2025. doi: 10.1145/3731569.3764843. URL https://madsys.cs.tsinghua.edu.cn/publication/ktransformers-unleashing-the-full-potential-of-cpu/gpu-hybrid-inference-for-moe-models/SOSP25-chen.pdf.

DeepSeek-AI. DeepSeek-V3 Technical Report, 2024. URL https://arxiv.org/abs/2412.19437.

DeepSeek-AI. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence, 2026a. URL https://arxiv.org/abs/2606.19348.

DeepSeek-AI. DeepSeek-V4-Flash-0731, 2026b. URL https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash-0731. Hugging Face model card, released 2026-07-31. Accessed 2026-11-06.

Fedus, W., Zoph, B., and Shazeer, N. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.

Gim, I., Chen, G., Lee, S.-s., Sarda, N., Khandelwal, A., and Zhong, L. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems 6 (MLSys)*, pp. 325–338, 2024.

URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html.

Hwang, R., Wei, J., Cao, S., Hwang, C., Tang, X., Cao, T., and Yang, M. Pre-gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Expert Inference. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 1018–1031. IEEE, 2024. doi: 10.1109/ISCA59077.2024.00078. URL https://doi.org/10.1109/ISCA59077.2024.00078.

ikawrakow. ik_llama.cpp: llama.cpp fork with additional SOTA quants and improved performance, 2026. URL https://github.com/ikawrakow/ik_llama.cpp. GitHub repository. Accessed 2026-11-07.

Jin, S., Liu, X., Zhang, Q., and Mao, Z. Compute or Load KV Cache? Why Not Both? In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pp. 28031–28043. PMLR, 2025. URL https://proceedings.mlr.press/v267/jin25d.html.

Kamahori, K., Tang, T., Gu, Y., Zhu, K., and Kasikci, B. Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/8cd1ce03ea58b3d7dfd809e4d42f08ea-Abstract-Conference.html.

kvcache-ai. KTransformers: A Flexible Framework for Experiencing Heterogeneous LLM Inference/Fine-tune Optimizations, 2026. URL https://github.com/kvcache-ai/ktransformers. GitHub repository, Apache-2.0 License. Accessed 2026-10-30.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pp. 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *International Conference on Learning Representations (ICLR)*, 2021. URL https://openreview.net/forum?id=qrwe7XHTmYb.

NVIDIA Corporation. CUDA Programming Guide, Section 4.2: CUDA Graphs, 2026. URL https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html. CUDA Programming Guide. Accessed 2026-10-30.

Oncescu, C.-A., Wu, Q., Chung, W. T., Wu, T.-C., Gopal, B. D., Wang, J., Dao, T., and Athiwaratkun, B. Opportunistic Expert Activation: Batch-Aware Expert Routing for Faster Decode Without Retraining. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pp. 94656–94674. PMLR, 2026. URL https://proceedings.mlr.press/v306/oncescu26a.html.

Pan, Z., Wang, M., Wu, C., Hu, Z., Qiang, X., Yu, Z., and Ding, Y. AgentReplay: Token-Wise Trace Replay Is Essential for Fair Serving System Performance Benchmarking, 2026. URL https://arxiv.org/abs/2609.32283.

Qwen Team. Qwen3-235B-A22B-Instruct-2507, 2025. URL https://huggingface.co/Qwen/Qwen3-235B-A22B-Instruct-2507. Hugging Face model card. Accessed 2026-10-30.

SamuelOliveirads. Add GPU cache for host MoE experts, 2026. URL https://github.com/ikawrakow/ik_llama.cpp/pull/2644. ik_llama.cpp pull request #2644. Accessed 2026-10-30.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1701.06538.

Song, X., Zhong, Z., Chen, R., and Chen, H. ProMoE: Fast MoE-based LLM Serving using Proactive Caching, 2024a. URL https://arxiv.org/abs/2410.22134.

Song, Y., Mi, Z., Xie, H., and Chen, H. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP)*, pp. 590–606. ACM, 2024b. doi: 10.1145/3694715.3695964. URL https://doi.org/10.1145/3694715.3695964.

spiritbuun. CUDA MoE expert cache, 2026. URL https://github.com/spiritbuun/buun-llama-cpp/blob/master/docs/backend/CUDA-MOE-CACHE.md. Documentation in the buun-llama-cpp repository. Accessed 2026-10-30.

Tang, P., Liu, J., Hou, X., Pu, Y., Wang, J., Heng, P.-A., Li, C., and Guo, M. HOBBIT: A Mixed Precision Expert Offloading System for Fast MoE Inference, 2024. URL https://arxiv.org/abs/2411.01433.

The ggml authors. llama.cpp: LLM inference in C/C++, 2026. URL https://github.com/ggml-org/llama.cpp. GitHub repository, MIT License. Accessed 2026-10-30.

Trofimova, M., Shevtsov, A., Badertdinov, I., Pyaev, K., Karasik, S., and Golubev, A. SWE-rebench-OpenHands-Trajectories, 2025. URL https://huggingface.co/datasets/nebius/SWE-rebench-openhands-trajectories. Hugging Face dataset. Accessed 2026-10-30.

Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, D., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., Brennan, R., Peng, H., Ji, H., and Neubig, G. Openhands: An open platform for ai software developers as generalist agents. In Yue, Y., Garg, A., Peng, N., Sha, F., and Yu, R. (eds.), *International Conference on Learning Representations*, volume 2025, pp. 65882–65919, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/a4b6ad6b48850c0c331d1259fc66a69c-Paper-Conference.pdf.

Wang, Y., Jin, Z., Chen, Z., Xu, J., Lin, W., Chen, Y., and Chen, W. AugServe: Adaptive Request Scheduling for Augmented Large Language Model Inference Serving. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pp. 127272–127292. PMLR, 2026a. URL https://proceedings.mlr.press/v306/wang26cu.html.

Wang, Z., Wang, Y., Gong, L., Tang, C., Lou, W., Wang, T., Wang, C., and Zhou, X. SeqMoE: Toward Full-Load Performance via Predictive and Graph-Compatible MoE Offloading, 2026b. URL https://arxiv.org/abs/2609.12978.

Xue, L., Fu, Y., Lu, Z., Mai, L., and Marina, M. MoE-Infinity: Efficient MoE Inference on Personal Machines with Sparsity-Aware Expert Cache, 2024. URL https://arxiv.org/abs/2401.14361.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 Technical Report, 2025. URL https://arxiv.org/abs/2505.09388.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, pp. 50528–50652, 2024. doi: 10.52202/079017-1601. URL https://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

Yang, S., Fan, X., Pan, M., Xi, H., Wang, Z., Sun, S., Keutzer, K., Han, S., Zaharia, M., Xu, C., and Stoica, I. FreeToken: Efficient Edge-Native MoE Serving with Bandwidth-Adaptive Execution, 2026a. URL https://arxiv.org/abs/2608.16157.

Yang, Y., Zhao, Y., Yang, P., Wang, S., and Zhou, Z.-H. ZipMoE: Efficient On-Device MoE Serving via Lossless Compression and Cache-Affinity Scheduling. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pp. 147261–147284. PMLR, 2026b. URL https://proceedings.mlr.press/v306/yang26ds.html.

Yu, E., Dong, D., Zhang, Z., Yang, X., Yang, W., Ma, S., Li, D., Wu, Y., and Liao, X. PDScope: Breaking the Prefill-Decode Seesaw for Memory-Bound MoE Inference, 2026. URL https://arxiv.org/abs/2610.39216. Accepted by PPoPP'27.

Yuan, Y., Nayak, A., Kundu, S., and Talati, N. Agentic AI Workload Characterization. In *2026 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2026. URL https://arxiv.org/abs/2605.26297.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, pp. 62557–62583, 2024. doi: 10.52202/079017-2000. URL https://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html.

Zhong, S., Sun, Y., Liang, L., Wang, R., Huang, R., and Li, M. HybriMoE: Hybrid CPU-GPU Scheduling and Cache Management for Efficient MoE Inference. In *2025 62nd*

ACM/IEEE Design Automation Conference (DAC), pp. 1–7. IEEE, 2025. doi: 10.1109/DAC63849.2025.11133274. URL https://arxiv.org/abs/2504.05897.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 193–210. USENIX Association, 2024.

Zhou, J., Dong, F., Huang, R., Cao, H., Chen, M., Yang, Y., Chen, A., Dong, M., Wang, Y., Li, D., Clifton, D. A., Lv, Q., Zhu, R., Zhang, C., Yang, F., Lu, T., Gu, N., and Shang, L. Oracle-MoE: Locality-preserving Routing in the Oracle Space for Memory-constrained Large Language Model Inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pp. 78633–78650. PMLR, 2025a. URL https://proceedings.mlr.press/v267/zhou25b.html.

Zhou, Y., Li, Z., Zhang, J., Wang, J., Wang, Y., Xie, Z., Chen, K., and Shou, L. FloE: On-the-Fly MoE Inference on Memory-constrained GPU. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pp. 78859–78882. PMLR, 2025b. URL https://proceedings.mlr.press/v267/zhou25j.html.

Zhu, K., Jacob, M., Ma, C., Pan, Y., Wang, S., Krishnamurthy, A., and Kasikci, B. TraceLab: Characterizing Coding Agent Workloads for LLM Serving, 2026a. URL https://arxiv.org/abs/2606.30560.

Zhu, X., Liao, X., Jiang, T., Zhang, Y., Wang, L., and Xiao, L. ReMoE: Boosting Expert Reuse through Router Fine-Tuning in Memory-Constrained MoE LLM Inference. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, volume 306 of *Proceedings of Machine Learning Research*, pp. 167311–167336. PMLR, 2026b. URL https://proceedings.mlr.press/v306/zhu26ab.html.

A. Correctness

OSTINATO changes where experts run, not what they compute. Under the same expert placement and routing, its decode path (the decode graph with borrowed memory) is bitwise identical to a synchronous reference that performs the same uploads without overlap; we compared both ways of issuing the uploads, whole experts and chunks, in a check window with 61 hits on borrowed experts, in a run that published 2,793 of them. The split and the upload path of prefill compute the same experts with the same weights through llama.cpp's native expert operators. Every held-out request of every engine replayed its recorded trajectory in full, returning the recorded tokens in every turn.[1]

B. Cost Model Derivations

Table 5 lists the notation. The results below hold for one-token decode steps, a fixed assignment of layers to GPUs and the costs calibrated inside the engine (Section 3.2).

Table 5. Notation of the cost model.

Symbol	Meaning
E, k	routed experts of an MoE layer; experts that a token selects
$\mathcal{L}_j$, κ_j	MoE layers on GPU j; expert slots of GPU j
H_ℓ, H	resident (hot) experts of layer ℓ; the residency set of all layers
r_ℓ	cold routes of layer ℓ in a decode step
G_ℓ	time of the GPU branch of layer ℓ in a decode step
$C(r)$	time of the CPU branch with r cold routes: γ_0 if $r = 0$, else $\alpha + \beta r$
g_ℓ, δ_ℓ	$\max\{\gamma_0, G_\ell\}$; $\alpha - g_\ell$
$\rho_{\ell e}$	routes to expert e of layer ℓ per decode step
$\mu_\ell(H)$, $q_\ell(H)$	expected cold routes of layer ℓ; probability that the layer has any
M, Q, F	$\sum_\ell \mu_\ell$; $\sum_\ell \delta_\ell q_\ell$; $\beta M + Q$
n, N_p	tokens of a prefill forward pass; prefill threshold
$r(n)$, $\bar{u}(n)$	cold routes and expected distinct cold experts of an n-token forward pass

Proposition B.1 (Expected time of an overlapped layer)**.** *Let layer ℓ have a fixed GPU-branch time G_ℓ and a CPU branch that takes $C(r) = \gamma_0$ for $r = 0$ cold routes and $\alpha + \beta r$ for $r \geq 1$. If $\beta \geq 0$ and $G_\ell \leq \alpha + \beta$, then*

$$\mathbb{E}\big[\max\{C(r_\ell), G_\ell\}\big] = g_\ell + \beta\mu_\ell + \delta_\ell\, q_\ell$$

for any joint distribution of the routes of a step.

Proof. If $r_\ell = 0$, the layer takes $\max\{\gamma_0, G_\ell\} = g_\ell$. If $r_\ell \geq 1$, then $\alpha + \beta r_\ell \geq \alpha + \beta \geq G_\ell$, so the layer takes $\alpha + \beta r_\ell$. Hence

$$\mathbb{E}\big[\max\{C(r_\ell), G_\ell\}\big] = (1 - q_\ell)\, g_\ell + \mathbb{E}\big[(\alpha + \beta r_\ell)\, \mathbf{1}\{r_\ell > 0\}\big] = (1 - q_\ell)\, g_\ell + \alpha q_\ell + \beta\mu_\ell,$$

which equals the claim and uses only the marginal distribution of r_ℓ. □

On DeepSeek-V4-Flash, $g_\ell = G_\ell$, and δ_ℓ lies between 0.0013 and 0.0056 ms, about 3% of β. With 4,864 resident experts, the development mean $\bar{q} = 0.589$ bounds $\sum_\ell \delta_\ell q_\ell$ by 0.142 ms per step, and for any residency set the term is at most $\sum_\ell \delta_\ell \approx 0.152$ ms.

Frequency-ranked residency. With $M(H) = \sum_\ell \mu_\ell(H)$ and $Q(H) = \sum_\ell \delta_\ell q_\ell(H)$, Proposition B.1 makes $F(H) = \beta M(H) + Q(H)$ the part of the expected decode-step time that depends on residency. Let H_F keep the κ_j most frequently routed experts of each GPU j, and let H_* minimize F. Proposition 3.1 states that $0 \leq F(H_F) - F(H_*) \leq Q(H_F) - Q(H_*) \leq Q(H_F)$.

Proof of Proposition 3.1. For every feasible H, $M(H) = \sum_{\ell,e} \rho_{\ell e} - \sum_\ell \sum_{e \in H_\ell} \rho_{\ell e}$. The first term does not depend on residency, and with equal-size slots and no per-layer limits, keeping the largest $\rho_{\ell e}$ on each GPU maximizes the second. Hence $M(H_F) \leq M(H_*)$, so H_F minimizes the expected number of cold routes, and

$$F(H_F) - F(H_*) = \beta\big(M(H_F) - M(H_*)\big) + Q(H_F) - Q(H_*) \leq Q(H_F) - Q(H_*) \leq Q(H_F),$$

[1] The top-k token selection of DeepSeek-V4-Flash's sparse-attention indexer breaks ties at the k-th place in arrival order, so two runs can select different but equally valid positions during prefill; with deterministic tie-breaking, runs reproduce bit for bit.

where the last step uses $\delta_\ell \geq 0$. The lower bound holds because H_* minimizes F. □

For a trace with N decode steps, let $z_{\ell eb}$ count the selections of expert e of layer ℓ in step b. Then $\rho_{\ell e} = N^{-1}\sum_b z_{\ell eb}$, $r_{\ell b}(H) = \sum_{e \notin H_\ell} z_{\ell eb}$ and $q_\ell(H) = N^{-1}\sum_b \mathbf{1}\{r_{\ell b}(H) > 0\}$, so joint activations enter q_ℓ through the trace, and no independence assumption is needed. On the development trace with 4,864 slots, $Q(H_F) \leq 0.142$ ms per step.

Slots of two sizes. When the experts of different layers differ in size, as in the Q4_K_M quantization of Qwen3-235B-A22B, the exchange argument holds within each size class. For a fixed split of a GPU's memory between the classes, each class keeps its most frequently routed experts, and the planner enumerates the feasible splits and keeps the one that covers the most development routes.

Proposition B.2 (Capacity increments)**.** *For nested residency sets $H \subseteq H'$, let $\Delta\mu_\ell = \mu_\ell(H) - \mu_\ell(H')$, $\Delta q_\ell = q_\ell(H) - q_\ell(H')$, $\Delta F = F(H) - F(H')$ and $\Delta F_0 = \beta\sum_\ell \Delta\mu_\ell$. If $\beta > 0$ and $0 \leq \delta_\ell \leq \delta_{\max}$, then $0 \leq \Delta q_\ell \leq \Delta\mu_\ell$ and*

$$\Delta F_0 \leq \Delta F \leq (1 + \delta_{\max}/\beta)\,\Delta F_0.$$

Proof. In every step, nesting gives $0 \leq r_\ell(H') \leq r_\ell(H)$, and because the counts are integers, $0 \leq \mathbf{1}\{r_\ell(H) > 0\} - \mathbf{1}\{r_\ell(H') > 0\} \leq r_\ell(H) - r_\ell(H')$. Taking expectations gives $0 \leq \Delta q_\ell \leq \Delta\mu_\ell$. Since $\Delta F = \Delta F_0 + \sum_\ell \delta_\ell\,\Delta q_\ell$, the second term lies between 0 and $\delta_{\max}\sum_\ell \Delta\mu_\ell = (\delta_{\max}/\beta)\,\Delta F_0$. □

On the development routes, the growth from 4,864 to 5,702 resident experts gives $\Delta F_0 = 2.219$ ms and $\Delta F = 2.239$ ms per step, within the factor $1 + 0.0056/0.184 \approx 1.03$; the held-out sessions measured 2.25 ms.

What overlap can hide. Because $C + G - \max\{C, G\} = \min\{C, G\}$, running the two branches concurrently hides at most $U = \sum_\ell \mathbb{E}[\min\{C(r_\ell), G_\ell\}]$ per step. On the same build and the same 160 decode steps, the step time falls from 38.415 to 33.538 ms when the branches overlap, a saving of 4.877 ms or 86.9% of $U = 5.615$ ms.

Lemma B.3 (Distinct cold experts)**.** *Let the routes of a prefill forward pass form a stationary sequence over its token positions, and let $\bar{u}(n)$ be the expected number of distinct cold experts that the first n positions select, with $\bar{u}(0) = 0$. Then $\bar{u}$ is nondecreasing, discretely concave and bounded by the number of cold experts.*

Proof. Let $A_{e,t}$ be the event that position t selects cold expert e. Then

$$\bar{u}(n+1) - \bar{u}(n) = \textstyle\sum_e \Pr\big(A_{e,n+1} \cap \bigcap_{t=1}^{n} A_{e,t}^c\big) \geq 0.$$

Dropping the condition on position 1 can only enlarge each event, and stationarity shifts the remaining positions by one, so

$$\bar{u}(n+2) - \bar{u}(n+1) \leq \textstyle\sum_e \Pr\big(A_{e,n+2} \cap \bigcap_{t=2}^{n+1} A_{e,t}^c\big) = \bar{u}(n+1) - \bar{u}(n).$$

No more than all cold experts can be selected, which gives the bound. □

Proof of Proposition 3.2. Write $\Phi(n)$ for the expected cost difference. By Lemma B.3, $\Phi(n+1) - \Phi(n) = \phi_1 - \phi_2\big(\bar{u}(n+1) - \bar{u}(n)\big)$ does not decrease with n, so Φ is discretely convex. Moreover, $\Phi(0) = \phi_0 < 0$, and $\Phi(n) \geq \phi_0 + \phi_1 n - \phi_2 c \to \infty$, where c is the number of cold experts. Let $n^* = \min\{n : \Phi(n) \geq 0\}$, so that $n^* \geq 1$ and $\Phi(n) < 0$ for $n < n^*$. The increment $\Phi(n^*) - \Phi(n^* - 1)$ is positive, and by convexity every later increment is at least as large, so $\Phi(n) > 0$ for $n > n^*$. □

Summing over layers preserves this form, so the whole forward pass also switches once. The two paths cost

$$\begin{aligned}\Pi^{\text{split}}(n) &= A_s(n) + C_n\big(r(n)\big) + G_h(n),\\ \Pi^{\text{up}}(n) &= A_u(n) + \big(w\,u(n) + X(n)\big)/B + G_h(n) + G_c(n) - \Omega(n),\end{aligned}$$

where A_s and A_u collect preparation and joins that are not overlapped, C_n is the CPU work on the cold routes, G_h and G_c are the GPU work on the hot and on the uploaded experts, X is the padding that the transfer carries and Ω is the overlap of uploads with computation. Their expected difference takes the form of Proposition 3.2, with $\phi_0 < 0$ the difference in setup cost, $\phi_1 > 0$ the CPU-minus-GPU cost of a token's cold routes and $\phi_2 = w/B$ the time to

upload one expert, when these terms grow linearly with the tokens and with the distinct cold experts; work that scales with the distinct cold experts on either path changes only the effective ϕ_2.

Calibration. We measure the CPU-branch time per layer inside the serving engine on development turns, for each number of cold routes. From $r = 1$ on, the time grows slightly less than linearly: each additional cold route adds 0.188 ms at $r = 2$ and 0.150 ms at $r = 6$. The fitted line $\alpha + \beta r$ lies up to 0.01 ms below the measured means for $r \leq 3$, which cover 98% of held-out layer steps, and 0.077 ms above the rare $r = 6$; a one-token step has at most $k = 6$ cold routes. On Qwen3-235B-A22B, whose 48 layers with 10.6 MB experts and 46 layers with 12.2 MB experts we calibrate separately, β is 0.139 and 0.157 ms, α is 0.128 and 0.129 ms and γ_0 is 0.118 and 0.119 ms; its GPU branches take 0.113 ms per layer on average, below γ_0, so $g_\ell = \gamma_0$ and the residual $\alpha - \gamma_0$, about 0.01 ms, is again small against β.

C. Configurations

All engines serve DeepSeek-V4-Flash from the same MXFP4 weights with a 98,304-token context, one slot and YaRN RoPE scaling with factor 16 over the original 65,536-token context. The llama.cpp-based engines run 48 threads on 48 physical cores, 24 on each socket, with host memory interleaved across the two sockets, a batch of 4,096 and a micro-batch of 2,048 tokens and an 8,192 MiB prompt cache. Table 6 lists what differs between the engines. KTransformers serves an SGLang-style HTTP interface that our replay client cannot drive, so we replay it with a separate lean client that timestamps the first and the last output token. Table 7 lists the median time to first token of every engine by turn type.

Table 6. Engine configurations on DeepSeek-V4-Flash. All engines use a 98,304-token context, one slot and YaRN RoPE scaling with factor 16; the four-GPU llama.cpp-based engines run 48 threads on 48 cores with a batch of 4,096 and a micro-batch of 2,048 tokens.

Engine	GPUs	Experts on the GPUs	Other settings
OSTINATO	4	5,702 most frequently routed experts in a static pool, plus 544 slots of borrowed memory during decode	prefill threshold N_p of 512 tokens
buun	4	dynamic cache of automatic size that admits experts by decaying access counts, warmed with a development routing profile	upload threshold of 512 tokens chosen on a development session; CPU–GPU overlap on
llama.cpp expert cache	4	74,048 MiB cache shared by the layers of each device	largest budget that passes the capacity test; dense weights placed by llama.cpp's automatic fit
ik_llama.cpp	4	69,632 MiB cache	budget chosen on development sessions
KTransformers	4	58 experts per layer in the default uniform placement, with dynamic expert updates	tensor parallelism over 4 GPUs; 48 CPU inference threads in two pools; prefill in chunks of 2,048 tokens; fastest of seven configurations on a development session
llama.cpp (layers)	4	whole MoE layers placed by llama.cpp's automatic fit, with 2,048 MiB left free on each GPU	no expert cache; forward passes of 512 or more tokens offload the CPU layers to the GPUs (threshold chosen on a development session)
OSTINATO	1	861 most frequently routed experts	32 threads on 32 cores, 16 on each socket; prefill threshold of 512 tokens
FreeToken	1	695 expert slots with device-to-device offload	32 CPU threads; prefill in chunks of 2,048 tokens

Table 7. OSTINATO has the shortest median time to first token after tool outputs and user messages; on first turns, llama.cpp's layer placement is 0.08–0.12 s faster. Median time to first token (s) by turn type on DeepSeek-V4-Flash, held-out sessions: the first turn of each SWE session or τ^2-bench task, the turns that follow a tool output and the τ^2-bench turns that follow a user message (number of turns in parentheses). Best values in bold.

	SWE		τ^2-bench		
Engine	first (13)	tool (880)	first (60)	tool (504)	user (352)
KTransformers	32.60	2.36	7.84	1.88	1.63
llama.cpp (layers)	**4.57**	1.66	**3.82**	1.25	0.47
llama.cpp expert cache	6.72	2.78	5.64	2.11	0.52
ik_llama.cpp	10.62	2.61	6.10	2.08	0.65
buun (llama.cpp fork)	5.82	2.37	5.33	1.80	0.73
OSTINATO	4.70	**1.26**	3.90	**0.98**	**0.37**

Client and server time. We measure the total request time at the client. For KTransformers, the total request time that the lean client measures lies within 0.18% and 0.14% of the end-to-end time that KTransformers itself reports on SWE and τ^2-bench. For the other pairs, the client and server measures give reductions within 0.5 percentage points of each other, except for ik_llama.cpp on τ^2-bench and buun on Qwen3-235B-A22B, whose reductions in server-side time are 35.6% and 14.9%, against 36.1% and 16.2% at the client; for buun on Qwen3, most of this difference is the start-up delay described below.

One-GPU comparison with FreeToken. Both engines use one RTX 4090D, the same 32 physical cores (16 on each socket) with 32 threads, memory interleaved across the two sockets and the official YaRN RoPE scaling with factor 16. OSTINATO keeps 861 experts resident and uses an upload threshold of 512 tokens; FreeToken runs its own GGUF path with 695 expert slots and its device-to-device offload. Each session ran twice on each engine, and we pair the means of the two runs. The two engines time prefill and decode differently, so we compare only total request times.

First turns on Qwen3-235B-A22B. Both engines run with the same prompt cache of 8,192 MiB, one slot and a 98,304-token context, and in every turn they reuse equally many tokens from the previous one. On the first request of each task, however, buun's time to first token exceeds its server-side prompt time by a median of 1.86 s, against 3 ms on its later requests, 0.25 s for OSTINATO and at most 3 ms for buun on DeepSeek-V4-Flash. Over the 60 first turns, the median time to first token is 0.77 s for OSTINATO and 3.17 s for buun, and the median prompt time is 0.48 against 1.19 s. We therefore report these turns here rather than in Table 4.

D. Ablation Numbers

Table 8. Ablation numbers behind Figure 8; each row adds one component to the row above. (a) Prefill and residency: total, prefill and decode time of the 60-turn screening set and of the 893 held-out SWE turns; these rows use the decode path of an earlier version, and the total also counts the time outside prefill and decode. (b) Decode: time per decode step on two development prefixes per workload; the prefill of each workload changes by less than 0.2 s across the rows.

(a) Prefill and residency

Configuration	Total (s)	Prefill (s)	Decode (s)
Screening set (60 turns)			
layer placement	629.1	165.6	463.7
llama.cpp expert cache	571.6	261.9	308.7
resident experts (5,702), native prefill	581.0	284.8	295.2
+ size-dependent prefill paths	447.7	150.8	295.8
Held-out SWE (893 turns)			
layer placement	12,084.9	1,779.8	10,269.0
llama.cpp expert cache	9,651.1	2,800.5	6,833.5
resident experts (4,864)	9,753.8	1,593.3	8,140.1
+ overlapped branches	8,679.3	1,590.4	7,071.3
+ 5,702 experts, prefill paths	8,116.6	1,511.2	6,588.3

(b) Decode, ms per step

Configuration	SWE	τ^2-bench
two branches, per-segment graphs	29.36	27.39
+ decode graph per GPU	27.43	25.45
+ no repeated affinity calls	27.01	25.21
+ borrowed memory (OSTINATO)	25.63	24.82
buun	28.65	27.95